

[NVIDIA DGX Spark](#)

How to Develop a CNN for MNIST Handwritten Digit Classification

by Jason Brownlee on November 14, 2021 in [Deep Learning for Computer Vision](#)  193

 Share  Share Post

How to Develop a Convolutional Neural Network From Scratch for MNIST Handwritten Digit Classification.

The MNIST handwritten digit classification problem is a standard dataset used in computer vision and deep learning.

Although the dataset is effectively solved, it can be used as the basis for learning and practicing how to develop, evaluate, and use convolutional deep learning neural networks for image classification from scratch. This includes how to develop a robust test harness for estimating the performance of the model, how to explore improvements to the model, and how to save the model and later load it to make predictions on new data.

In this tutorial, you will discover how to develop a convolutional neural network for handwritten digit classification from scratch.

After completing this tutorial, you will know:

- How to develop a test harness to develop a robust evaluation of a model and establish a baseline of performance for a classification task.
- How to explore extensions to a baseline model to improve learning and model capacity.
- How to develop a finalized model, evaluate the performance of the final model, and use it to make predictions on new images.

Kick-start your project with my new book [Deep Learning for Computer Vision](#), including *step-by-step tutorials* and the *Python source code* files for all examples.

Let's get started.

- **Updated Dec/2019:** Updated examples for TensorFlow 2.0 and Keras 2.3.
- **Updated Jan/2020:** Fixed a bug where models were defined outside the cross-validation loop.
- **Updated Nov/2021:** Updated to use Tensorflow 2.6



How to Develop a Convolutional Neural Network From Scratch for MNIST Handwritten Digit Classification
Photo by Richard Allaway, some rights reserved.

Tutorial Overview

This tutorial is divided into five parts; they are:

1. MNIST Handwritten Digit Classification Dataset
2. Model Evaluation Methodology
3. How to Develop a Baseline Model

4. How to Develop an Improved Model
5. How to Finalize the Model and Make Predictions

Want Results with Deep Learning for Computer Vision?

Take my free 7-day email crash course now (with sample code).

Click to sign-up and also get a free PDF Ebook version of the course.

Click here to subscribe

Development Environment

This tutorial assumes that you are using standalone Keras running on top of TensorFlow with Python 3. If you need help setting up your development environment see this tutorial:

- [How to Setup Your Python Environment for Machine Learning with Anaconda](#)

MNIST Handwritten Digit Classification Dataset

The **MNIST dataset** is an acronym that stands for the Modified National Institute of Standards and Technology dataset.

It is a dataset of 60,000 small square 28×28 pixel grayscale images of handwritten single digits between 0 and 9.

The task is to classify a given image of a handwritten digit into one of 10 classes representing integer values from 0 to 9, inclusively.

It is a widely used and deeply understood dataset and, for the most part, is “solved.” Top-performing models are deep learning convolutional neural networks that achieve a classification accuracy of above 99%, with an error rate between 0.4 %and 0.2% on the hold out test dataset.

The example below loads the MNIST dataset using the Keras API and creates a plot of the first nine images in the training dataset.

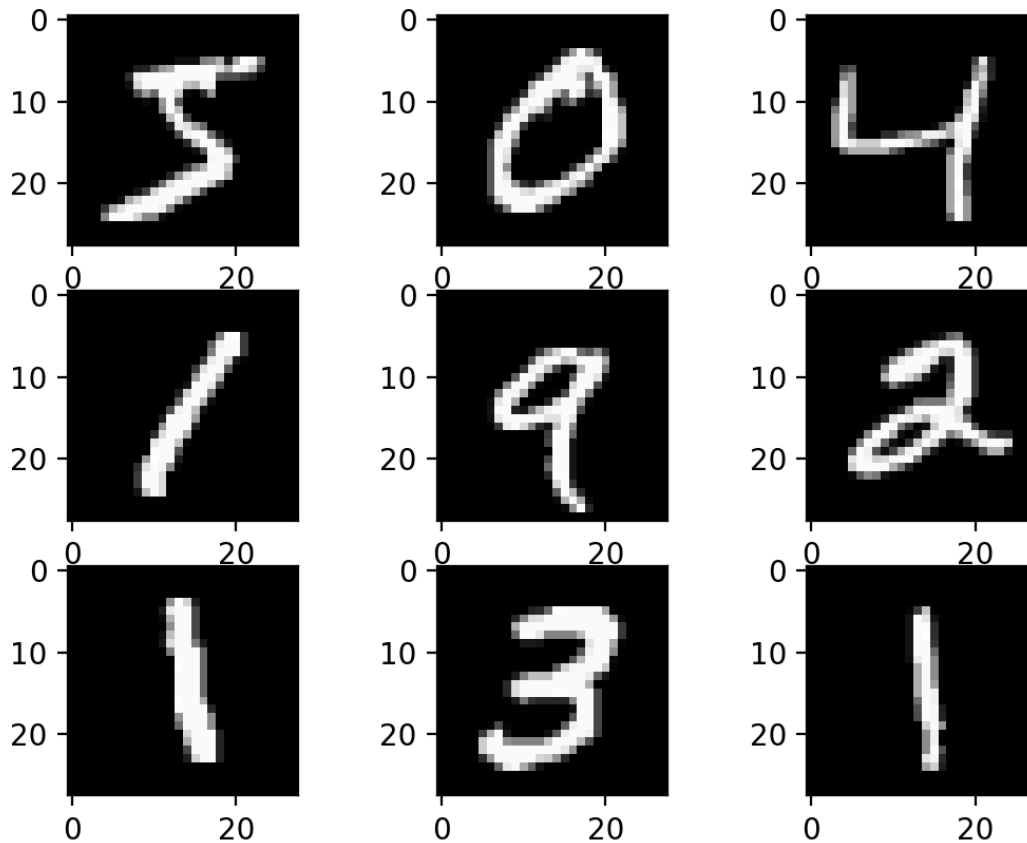
```
1 # example of loading the mnist dataset
2 from tensorflow.keras.datasets import mnist
3 from matplotlib import pyplot as plt
4 # load dataset
5 (trainX, trainy), (testX, testy) = mnist.load_data()
6 # summarize loaded dataset
7 print('Train: X=%s, y=%s' % (trainX.shape, trainy.shape))
8 print('Test: X=%s, y=%s' % (testX.shape, testy.shape))
9 # plot first few images
10 for i in range(9):
11     # define subplot
12     plt.subplot(330 + 1 + i)
13     # plot raw pixel data
14     plt.imshow(trainX[i], cmap=plt.get_cmap('gray'))
15 # show the figure
16 plt.show()
```

Running the example loads the MNIST train and test dataset and prints their shape.

We can see that there are 60,000 examples in the training dataset and 10,000 in the test dataset and that images are indeed square with 28×28 pixels.

```
1 Train: X=(60000, 28, 28), y=(60000,)
2 Test: X=(10000, 28, 28), y=(10000,)
```

A plot of the first nine images in the dataset is also created showing the natural handwritten nature of the images to be classified.



Plot of a Subset of Images From the MNIST Dataset

Model Evaluation Methodology

Although the MNIST dataset is effectively solved, it can be a useful starting point for developing and practicing a methodology for solving image classification tasks using convolutional neural networks.

Instead of reviewing the literature on well-performing models on the dataset, we can develop a new model from scratch.

The dataset already has a well-defined train and test dataset that we can use.

In order to estimate the performance of a model for a given training run, we can further split the training set into a train and validation dataset. Performance on the train and validation dataset over each run can then be plotted to provide learning curves and insight into how well a model is learning the problem.

The Keras API supports this by specifying the “*validation_data*” argument to the *model.fit()* function when training the model, that will, in turn, return an object that describes model performance for the chosen loss and metrics on each training epoch.

```
1 # record model performance on a validation dataset during training
2 history = model.fit(..., validation_data=(valX, valY))
```

In order to estimate the performance of a model on the problem in general, we can use **k-fold cross-validation**, perhaps five-fold cross-validation. This will give some account of the model's variance with both respect to differences in the training and test datasets, and in terms of the stochastic nature of the learning algorithm. The performance of a model can be taken as the mean performance across k-folds, given the standard deviation, that could be used to estimate a confidence interval if desired.

We can use the **KFold** class from the scikit-learn API to implement the k-fold cross-validation evaluation of a given neural network model. There are many ways to achieve this, although we can choose a flexible approach where the **KFold** class is only used to specify the row indexes used for each split.

```
1 # example of k-fold cv for a neural net
2 data = ...
3 # prepare cross validation
4 kf = KFold(5, shuffle=True, random_state=1)
5 # enumerate splits
6 for train_ix, test_ix in kf.split(data):
7     model = ...
8     ...
```

We will hold back the actual test dataset and use it as an evaluation of our final model.

How to Develop a Baseline Model

The first step is to develop a baseline model.

This is critical as it both involves developing the infrastructure for the test harness so that any model we design can be evaluated on the dataset, and it establishes a baseline in model performance on the problem, by which all improvements can be compared.

The design of the test harness is modular, and we can develop a separate function for each piece. This allows a given aspect of the test harness to be modified or inter-changed, if we desire, separately from the rest.

We can develop this test harness with five key elements. They are the loading of the dataset, the preparation of the dataset, the definition of the model, the evaluation of the model, and the presentation of results.

Load Dataset

We know some things about the dataset.

For example, we know that the images are all pre-aligned (e.g. each image only contains a hand-drawn digit), that the images all have the same square size of 28×28 pixels, and that the images are grayscale.

Therefore, we can load the images and reshape the data arrays to have a single color channel.

```
1 # load dataset
2 (trainX, trainY), (testX, testY) = mnist.load_data()
3 # reshape dataset to have a single channel
4 trainX = trainX.reshape((trainX.shape[0], 28, 28, 1))
5 testX = testX.reshape((testX.shape[0], 28, 28, 1))
```

We also know that there are 10 classes and that classes are represented as unique integers.

We can, therefore, use a one hot encoding for the class element of each sample, transforming the integer into a 10 element binary vector with a 1 for the index of the class value, and 0 values for all other classes. We can achieve this with the `to_categorical()` utility function.

```
1 # one hot encode target values
2 trainY = to_categorical(trainY)
3 testY = to_categorical(testY)
```

The `load_dataset()` function implements these behaviors and can be used to load the dataset.

```
1 # load train and test dataset
2 def load_dataset():
3     # load dataset
4     (trainX, trainY), (testX, testY) = mnist.load_data()
5     # reshape dataset to have a single channel
6     trainX = trainX.reshape((trainX.shape[0], 28, 28, 1))
7     testX = testX.reshape((testX.shape[0], 28, 28, 1))
8     # one hot encode target values
9     trainY = to_categorical(trainY)
10    testY = to_categorical(testY)
11    return trainX, trainY, testX, testY
```

Prepare Pixel Data

We know that the pixel values for each image in the dataset are unsigned integers in the range between black and white, or 0 and 255.

We do not know the best way to scale the pixel values for modeling, but we know that some scaling will be required.

A good starting point is to **normalize the pixel values** of grayscale images, e.g. rescale them to the range [0,1]. This involves first converting the data type from unsigned integers to floats, then dividing the pixel values by the maximum value.

```
1 # convert from integers to floats
2 train_norm = train.astype('float32')
3 test_norm = test.astype('float32')
4 # normalize to range 0-1
5 train_norm = train_norm / 255.0
6 test_norm = test_norm / 255.0
```

The `prep_pixels()` function below implements these behaviors and is provided with the pixel values for both the train and test datasets that will need to be scaled.

```
1 # scale pixels
2 def prep_pixels(train, test):
3     # convert from integers to floats
4     train_norm = train.astype('float32')
5     test_norm = test.astype('float32')
6     # normalize to range 0-1
7     train_norm = train_norm / 255.0
8     test_norm = test_norm / 255.0
9     # return normalized images
10    return train_norm, test_norm
```

This function must be called to prepare the pixel values prior to any modeling.

Define Model

Next, we need to define a baseline convolutional neural network model for the problem.

The model has two main aspects: the feature extraction front end comprised of convolutional and pooling layers, and the classifier backend that will make a prediction.

For the convolutional front-end, we can start with a single [convolutional layer](#) with a small filter size (3,3) and a modest number of filters (32) followed by a [max pooling layer](#). The filter maps can then be flattened to provide features to the classifier.

Given that the problem is a multi-class classification task, we know that we will require an output layer with 10 nodes in order to predict the probability distribution of an image belonging to each of the 10 classes. This will also require the use of a softmax activation function. Between the feature extractor and the output layer, we can add a dense layer to interpret the features, in this case with 100 nodes.

All layers will use the [ReLU activation function](#) and the He weight initialization scheme, both best practices.

We will use a conservative configuration for the stochastic gradient descent optimizer with a [learning rate](#) of 0.01 and a momentum of 0.9. The [categorical cross-entropy](#) loss function will be optimized, suitable for multi-class classification, and we will monitor the classification accuracy metric, which is appropriate given we have the same number of examples in each of the 10 classes.

The `define_model()` function below will define and return this model.

```
1 # define cnn model
2 def define_model():
3     model = Sequential()
4     model.add(Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_uniform', input_shape=(28, 28, 1)))
5     model.add(MaxPooling2D((2, 2)))
6     model.add(Flatten())
7     model.add(Dense(100, activation='relu', kernel_initializer='he_uniform'))
8     model.add(Dense(10, activation='softmax'))
9     # compile model
10    opt = SGD(learning_rate=0.01, momentum=0.9)
11    model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['accuracy'])
12    return model
```

Evaluate Model

After the model is defined, we need to evaluate it.

The model will be evaluated using [five-fold cross-validation](#). The value of $k=5$ was chosen to provide a baseline for both repeated evaluation and to not be so large as to require a long running time. Each test set will be 20% of the training dataset, or about 12,000 examples, close to the size of the actual test set for this problem.

The training dataset is shuffled prior to being split, and the sample shuffling is performed each time, so that any model we evaluate will have the same train and test datasets in each fold, providing an apples-to-apples comparison between models.

We will train the baseline model for a modest 10 training epochs with a default batch size of 32 examples. The test set for each fold will be used to evaluate the model both during each epoch of the training run, so that we can later create learning curves, and at the end of the run, so that we can estimate the performance of the model. As such, we will keep track of the resulting history from each run, as well as the classification accuracy of the fold.

The `evaluate_model()` function below implements these behaviors, taking the training dataset as arguments and returning a list of accuracy scores and training histories that can be later summarized.

```
1 # evaluate a model using k-fold cross-validation
2 def evaluate_model(dataX, dataY, n_folds=5):
3     scores, histories = list(), list()
4     # prepare cross validation
5     kfold = KFold(n_folds, shuffle=True, random_state=1)
6     # enumerate splits
7     for train_ix, test_ix in kfold.split(dataX):
8         # define model
9         model = define_model()
10        # select rows for train and test
11        trainX, trainY, testX, testY = dataX[train_ix], dataY[train_ix], dataX[test_ix], dataY[test_ix]
12        # fit model
13        history = model.fit(trainX, trainY, epochs=10, batch_size=32, validation_data=(testX, testY), verbose=0)
14        # evaluate model
15        _, acc = model.evaluate(testX, testY, verbose=0)
16        print('> %.3f' % (acc * 100.0))
17        # stores scores
18        scores.append(acc)
19        histories.append(history)
20    return scores, histories
```

Present Results

Once the model has been evaluated, we can present the results.

There are two key aspects to present: the diagnostics of the learning behavior of the model during training and the estimation of the model performance. These can be implemented using separate functions.

First, the diagnostics involve creating a line plot showing model performance on the train and test set during each fold of the k-fold cross-validation. These plots are valuable for getting an idea of whether a model is overfitting, underfitting, or has a good fit for the dataset.

We will create a single figure with two subplots, one for loss and one for accuracy. Blue lines will indicate model performance on the training dataset and orange lines will indicate performance on the hold out test dataset. The `summarize_diagnostics()` function below creates and shows this plot given the collected training histories.

```
1 # plot diagnostic learning curves
2 def summarize_diagnostics(histories):
3     for i in range(len(histories)):
4         # plot loss
5         plt.subplot(2, 1, 1)
6         plt.title('Cross Entropy Loss')
7         plt.plot(histories[i].history['loss'], color='blue', label='train')
8         plt.plot(histories[i].history['val_loss'], color='orange', label='test')
9         # plot accuracy
10        plt.subplot(2, 1, 2)
11        plt.title('Classification Accuracy')
12        plt.plot(histories[i].history['accuracy'], color='blue', label='train')
13        plt.plot(histories[i].history['val_accuracy'], color='orange', label='test')
14    plt.show()
```

Next, the classification accuracy scores collected during each fold can be summarized by calculating the mean and standard deviation. This provides an estimate of the average expected performance of the model trained on this dataset, with an estimate of the average variance in the mean. We will also summarize the distribution of scores by creating and showing a box and whisker plot.

The `summarize_performance()` function below implements this for a given list of scores collected during model evaluation.

```
1 # summarize model performance
2 def summarize_performance(scores):
3     # print summary
4     print('Accuracy: mean=%.3f std=%.3f, n=%d' % (mean(scores)*100, std(scores)*100, len(scores)))
5     # box and whisker plots of results
6     plt.boxplot(scores)
7     plt.show()
```

Complete Example

We need a function that will drive the test harness.

This involves calling all of the define functions.

```
1 # run the test harness for evaluating a model
2 def run_test_harness():
3     # load dataset
4     trainX, trainY, testX, testY = load_dataset()
5     # prepare pixel data
6     trainX, testX = prep_pixels(trainX, testX)
7     # evaluate model
8     scores, histories = evaluate_model(trainX, trainY)
9     # learning curves
10    summarize_diagnostics(histories)
11    # summarize estimated performance
12    summarize_performance(scores)
```

We now have everything we need; the complete code example for a baseline convolutional neural network model on the MNIST dataset is listed below.

```
1 # baseline cnn model for mnist
2 from numpy import mean
3 from numpy import std
4 from matplotlib import pyplot as plt
5 from sklearn.model_selection import KFold
6 from tensorflow.keras.datasets import mnist
7 from tensorflow.keras.utils import to_categorical
8 from tensorflow.keras.models import Sequential
9 from tensorflow.keras.layers import Conv2D
10 from tensorflow.keras.layers import MaxPooling2D
11 from tensorflow.keras.layers import Dense
12 from tensorflow.keras.layers import Flatten
13 from tensorflow.keras.optimizers import SGD
14
15 # load train and test dataset
16 def load_dataset():
17     # load dataset
18     (trainX, trainY), (testX, testY) = mnist.load_data()
19     # reshape dataset to have a single channel
20     trainX = trainX.reshape((trainX.shape[0], 28, 28, 1))
21     testX = testX.reshape((testX.shape[0], 28, 28, 1))
22     # one hot encode target values
23     trainY = to_categorical(trainY)
24     testY = to_categorical(testY)
25     return trainX, trainY, testX, testY
26
27 # scale pixels
28 def prep_pixels(train, test):
29     # convert from integers to floats
30     train_norm = train.astype('float32')
31     test_norm = test.astype('float32')
```



```

32     # normalize to range 0-1
33     train_norm = train_norm / 255.0
34     test_norm = test_norm / 255.0
35     # return normalized images
36     return train_norm, test_norm
37
38 # define cnn model
39 def define_model():
40     model = Sequential()
41     model.add(Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_uniform', input_shape=(28, 28, 1)))
42     model.add(MaxPooling2D((2, 2)))
43     model.add(Flatten())
44     model.add(Dense(100, activation='relu', kernel_initializer='he_uniform'))
45     model.add(Dense(10, activation='softmax'))
46     # compile model
47     opt = SGD(learning_rate=0.01, momentum=0.9)
48     model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['accuracy'])
49     return model
50
51 # evaluate a model using k-fold cross-validation
52 def evaluate_model(dataX, dataY, n_folds=5):
53     scores, histories = list(), list()
54     # prepare cross validation
55     kf = KFold(n_folds, shuffle=True, random_state=1)
56     # enumerate splits
57     for train_ix, test_ix in kf.split(dataX):
58         # define model
59         model = define_model()
60         # select rows for train and test
61         trainX, trainY, testX, testY = dataX[train_ix], dataY[train_ix], dataX[test_ix], dataY[test_ix]
62         # fit model
63         history = model.fit(trainX, trainY, epochs=10, batch_size=32, validation_data=(testX, testY), verbose=0)
64         # evaluate model
65         _, acc = model.evaluate(testX, testY, verbose=0)
66         print('> %.3f' % (acc * 100.0))
67         # store scores
68         scores.append(acc)
69         histories.append(history)
70     return scores, histories
71
72 # plot diagnostic learning curves
73 def summarize_diagnostics(histories):
74     for i in range(len(histories)):
75         # plot loss
76         plt.subplot(2, 1, 1)
77         plt.title('Cross Entropy Loss')
78         plt.plot(histories[i].history['loss'], color='blue', label='train')
79         plt.plot(histories[i].history['val_loss'], color='orange', label='test')
80         # plot accuracy
81         plt.subplot(2, 1, 2)
82         plt.title('Classification Accuracy')
83         plt.plot(histories[i].history['accuracy'], color='blue', label='train')
84         plt.plot(histories[i].history['val_accuracy'], color='orange', label='test')
85     plt.show()
86
87 # summarize model performance
88 def summarize_performance(scores):
89     # print summary
90     print('Accuracy: mean=%.3f std=%.3f, n=%d' % (mean(scores)*100, std(scores)*100, len(scores)))
91     # box and whisker plots of results
92     plt.boxplot(scores)
93     plt.show()
94
95 # run the test harness for evaluating a model
96 def run_test_harness():
97     # load dataset
98     trainX, trainY, testX, testY = load_dataset()
99     # prepare pixel data
100    trainX, testX = prep_pixels(trainX, testX)
101    # evaluate model
102    scores, histories = evaluate_model(trainX, trainY)
103    # learning curves
104    summarize_diagnostics(histories)
105    # summarize estimated performance
106    summarize_performance(scores)
107
108 # entry point, run the test harness
109 run_test_harness()

```

Running the example prints the classification accuracy for each fold of the cross-validation process. This is helpful to get an idea that the model evaluation is progressing.

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

We can see two cases where the model achieves perfect skill and one case where it achieved lower than 98% accuracy. These are good results.

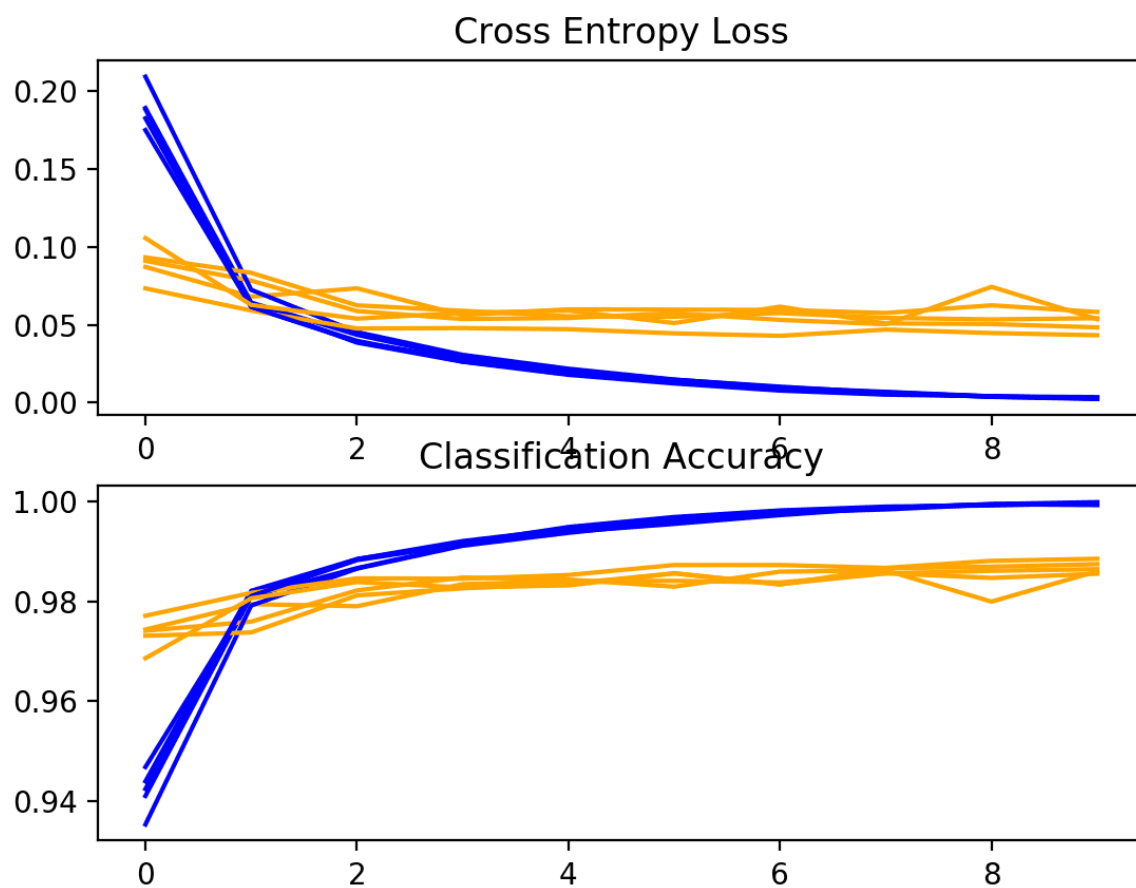
```

1 > 98.550
2 > 98.600
3 > 98.642
4 > 98.850
5 > 98.742

```

Next, a diagnostic plot is shown, giving insight into the learning behavior of the model across each fold.

In this case, we can see that the model generally achieves a good fit, with train and test learning curves converging. There is no obvious sign of over- or underfitting.



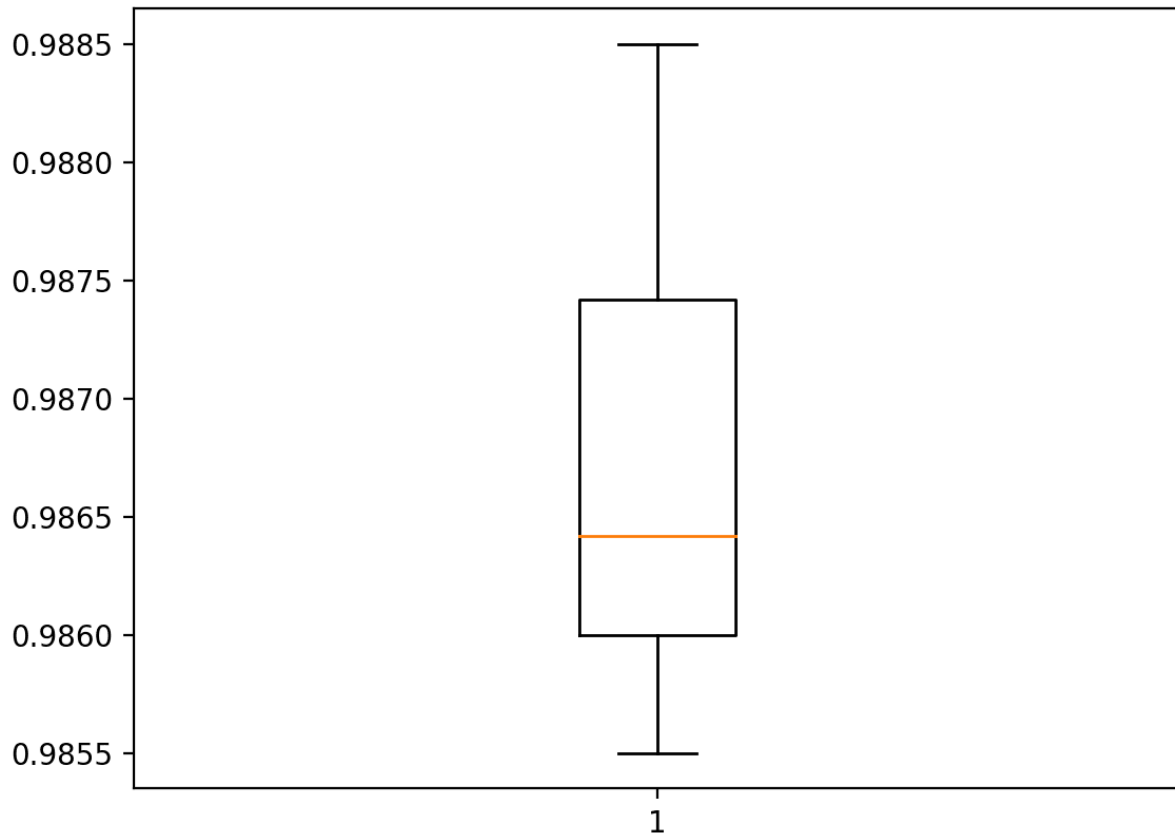
Loss and Accuracy Learning Curves for the Baseline Model During k-Fold Cross-Validation

Next, a summary of the model performance is calculated.

We can see in this case, the model has an estimated skill of about 98.6%, which is reasonable.

```
1 Accuracy: mean=98.677 std=0.107, n=5
```

Finally, a box and whisker plot is created to summarize the distribution of accuracy scores.



Box and Whisker Plot of Accuracy Scores for the Baseline Model Evaluated Using k-Fold Cross-Validation

We now have a robust test harness and a well-performing baseline model.

How to Develop an Improved Model

There are many ways that we might explore improvements to the baseline model.

We will look at areas of model configuration that often result in an improvement, so-called low-hanging fruit. The first is a change to the learning algorithm, and the second is an increase in the depth of the model.

Improvement to Learning

There are many aspects of the learning algorithm that can be explored for improvement.

Perhaps the point of biggest leverage is the learning rate, such as evaluating the impact that smaller or larger values of the learning rate may have, as well as schedules that change the learning rate during training.

Another approach that can rapidly accelerate the learning of a model and can result in large performance improvements is batch normalization. We will evaluate the effect that batch normalization has on our baseline model.

Batch normalization can be used after convolutional and fully connected layers. It has the effect of changing the distribution of the output of the layer, specifically by standardizing the outputs. This has the effect of stabilizing and accelerating the learning process.

We can update the model definition to use batch normalization after the activation function for the convolutional and dense layers of our baseline model. The updated version of `define_model()` function with batch normalization is listed below.

```

1 # define cnn model
2 def define_model():
3     model = Sequential()
4     model.add(Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_uniform', input_shape=(28, 28, 1)))
5     model.add(BatchNormalization())
6     model.add(MaxPooling2D((2, 2)))
7     model.add(Flatten())
8     model.add(Dense(100, activation='relu', kernel_initializer='he_uniform'))
9     model.add(BatchNormalization())
10    model.add(Dense(10, activation='softmax'))
11    # compile model
12    opt = SGD(learning_rate=0.01, momentum=0.9)

```

```

13     model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['accuracy'])
14     return model

```

The complete code listing with this change is provided below.

```

1  # cnn model with batch normalization for mnist
2  from numpy import mean
3  from numpy import std
4  from matplotlib import pyplot as plt
5  from sklearn.model_selection import KFold
6  from tensorflow.keras.datasets import mnist
7  from tensorflow.keras.utils import to_categorical
8  from tensorflow.keras.models import Sequential
9  from tensorflow.keras.layers import Conv2D
10 from tensorflow.keras.layers import MaxPooling2D
11 from tensorflow.keras.layers import Dense
12 from tensorflow.keras.layers import Flatten
13 from tensorflow.keras.optimizers import SGD
14 from tensorflow.keras.layers import BatchNormalization
15
16 # load train and test dataset
17 def load_dataset():
18     # load dataset
19     (trainX, trainY), (testX, testY) = mnist.load_data()
20     # reshape dataset to have a single channel
21     trainX = trainX.reshape((trainX.shape[0], 28, 28, 1))
22     testX = testX.reshape((testX.shape[0], 28, 28, 1))
23     # one hot encode target values
24     trainY = to_categorical(trainY)
25     testY = to_categorical(testY)
26     return trainX, trainY, testX, testY
27
28 # scale pixels
29 def prep_pixels(train, test):
30     # convert from integers to floats
31     train_norm = train.astype('float32')
32     test_norm = test.astype('float32')
33     # normalize to range 0-1
34     train_norm = train_norm / 255.0
35     test_norm = test_norm / 255.0
36     # return normalized images
37     return train_norm, test_norm
38
39 # define cnn model
40 def define_model():
41     model = Sequential()
42     model.add(Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_uniform', input_shape=(28, 28, 1)))
43     model.add(BatchNormalization())
44     model.add(MaxPooling2D((2, 2)))
45     model.add(Flatten())
46     model.add(Dense(100, activation='relu', kernel_initializer='he_uniform'))
47     model.add(BatchNormalization())
48     model.add(Dense(10, activation='softmax'))
49     # compile model
50     opt = SGD(learning_rate=0.01, momentum=0.9)
51     model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['accuracy'])
52     return model
53
54 # evaluate a model using k-fold cross-validation
55 def evaluate_model(dataX, dataY, n_folds=5):
56     scores, histories = list(), list()
57     # prepare cross validation
58     kfold = KFold(n_folds, shuffle=True, random_state=1)
59     # enumerate splits
60     for train_ix, test_ix in kfold.split(dataX):
61         # define model
62         model = define_model()
63         # select rows for train and test
64         trainX, trainY, testX, testY = dataX[train_ix], dataY[train_ix], dataX[test_ix], dataY[test_ix]
65         # fit model
66         history = model.fit(trainX, trainY, epochs=10, batch_size=32, validation_data=(testX, testY), verbose=0)
67         # evaluate model
68         _, acc = model.evaluate(testX, testY, verbose=0)
69         print('> %.3f' % (acc * 100.0))
70         # stores scores
71         scores.append(acc)
72         histories.append(history)
73     return scores, histories
74
75 # plot diagnostic learning curves
76 def summarize_diagnostics(histories):
77     for i in range(len(histories)):
78         # plot loss
79         plt.subplot(2, 1, 1)
80         plt.title('Cross Entropy Loss')
81         plt.plot(histories[i].history['loss'], color='blue', label='train')
82         plt.plot(histories[i].history['val_loss'], color='orange', label='test')
83         # plot accuracy
84         plt.subplot(2, 1, 2)
85         plt.title('Classification Accuracy')
86         plt.plot(histories[i].history['accuracy'], color='blue', label='train')
87         plt.plot(histories[i].history['val_accuracy'], color='orange', label='test')
88     plt.show()
89
90 # summarize model performance
91 def summarize_performance(scores):
92     # print summary
93     print('Accuracy: mean=%.3f std=%.3f, n=%d' % (mean(scores)*100, std(scores)*100, len(scores)))
94     # box and whisker plots of results
95     plt.boxplot(scores)
96     plt.show()
97
98 # run the test harness for evaluating a model

```

```

99 def run_test_harness():
100     # load dataset
101     trainX, trainY, testX, testY = load_dataset()
102     # prepare pixel data
103     trainX, testX = prep_pixels(trainX, testX)
104     # evaluate model
105     scores, histories = evaluate_model(trainX, trainY)
106     # learning curves
107     summarize_diagnostics(histories)
108     # summarize estimated performance
109     summarize_performance(scores)
110
111 # entry point, run the test harness
112 run_test_harness()

```

Running the example again reports model performance for each fold of the cross-validation process.

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

We can see perhaps a small drop in model performance as compared to the baseline across the cross-validation folds.

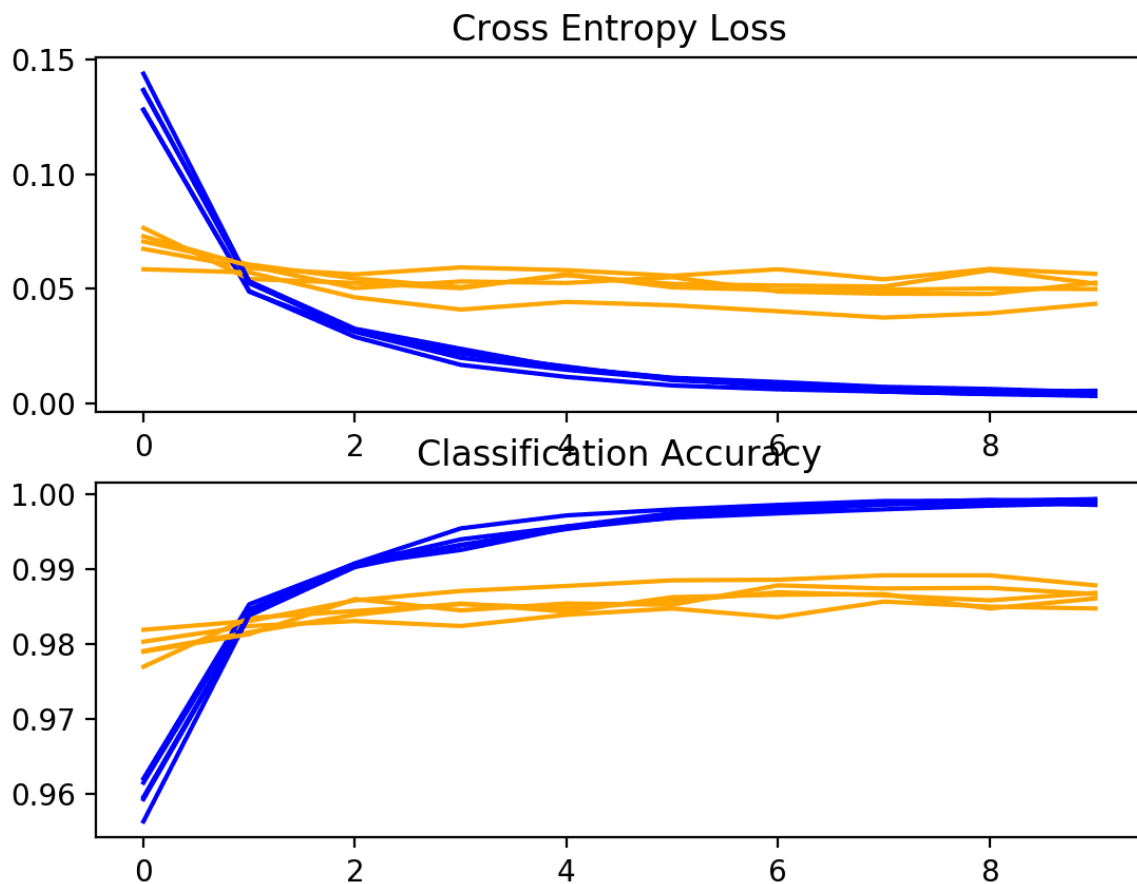
```

1 > 98.475
2 > 98.608
3 > 98.683
4 > 98.783
5 > 98.667

```

A plot of the learning curves is created, in this case showing that the speed of learning (improvement over epochs) does not appear to be different from the baseline model.

The plots suggest that batch normalization, at least as implemented in this case, does not offer any benefit.

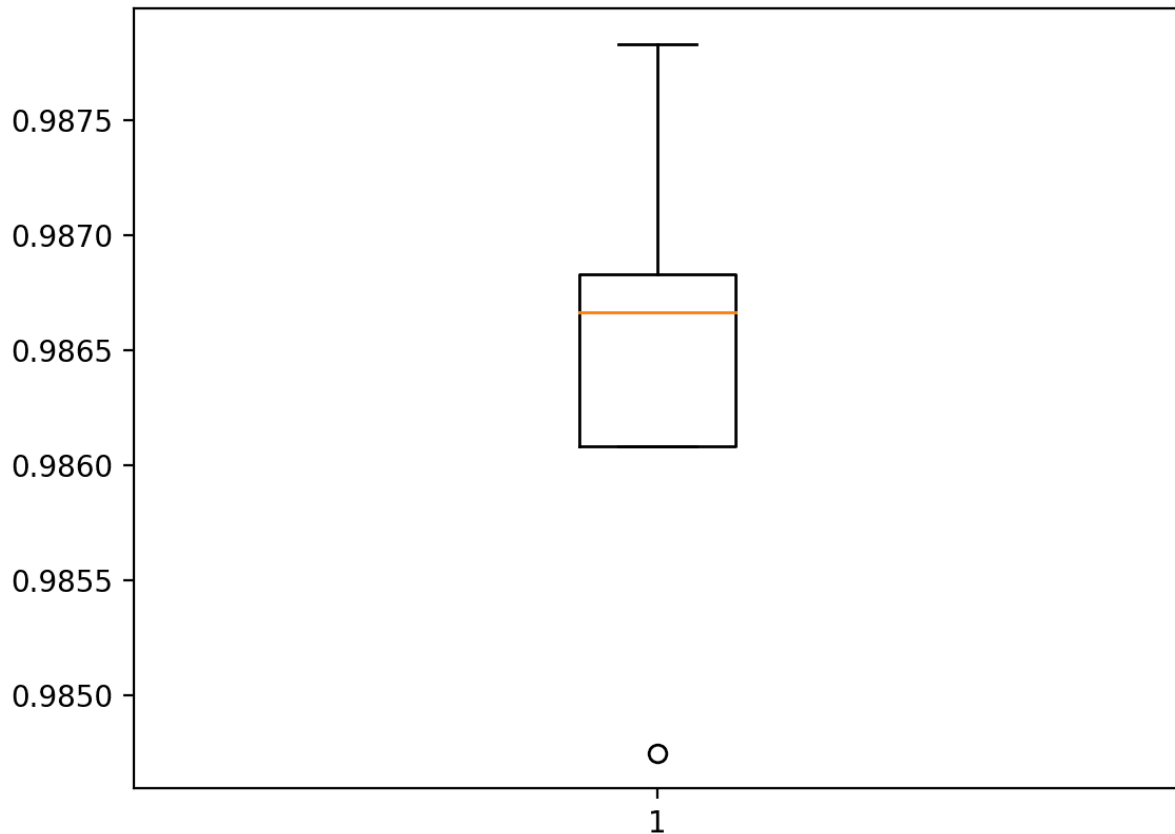


Next, the estimated performance of the model is presented, showing performance with a slight decrease in the mean accuracy of the model: 98.643 as compared to 98.677 with the baseline model.

```

1 Accuracy: mean=98.643 std=0.101, n=5

```



Box and Whisker Plot of Accuracy Scores for the BatchNormalization Model Evaluated Using k-Fold Cross-Validation

Increase in Model Depth

There are many ways to change the model configuration in order to explore improvements over the baseline model.

Two common approaches involve changing the **capacity** of the feature extraction part of the model or changing the capacity or function of the classifier part of the model. Perhaps the point of biggest influence is a change to the feature extractor.

We can increase the depth of the feature extractor part of the model, following a VGG-like **pattern** of adding more convolutional and pooling layers with the same sized filter, while increasing the number of filters. In this case, we will add a double convolutional layer with 64 filters each, followed by another max pooling layer.

The updated version of the `define_model()` function with this change is listed below.

```
1 # define cnn model
2 def define_model():
3     model = Sequential()
4     model.add(Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_uniform', input_shape=(28, 28, 1)))
5     model.add(MaxPooling2D((2, 2)))
6     model.add(Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_uniform'))
7     model.add(Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_uniform'))
8     model.add(MaxPooling2D((2, 2)))
9     model.add(Flatten())
10    model.add(Dense(100, activation='relu', kernel_initializer='he_uniform'))
11    model.add(Dense(10, activation='softmax'))
12    # compile model
13    opt = SGD(learning_rate=0.01, momentum=0.9)
14    model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['accuracy'])
15    return model
```

For completeness, the entire code listing, including this change, is provided below.

```
1 # deeper cnn model for mnist
2 from numpy import mean
3 from numpy import std
4 from matplotlib import pyplot as plt
5 from sklearn.model_selection import KFold
6 from tensorflow.keras.datasets import mnist
7 from tensorflow.keras.utils import to_categorical
8 from tensorflow.keras.models import Sequential
9 from tensorflow.keras.layers import Conv2D
10 from tensorflow.keras.layers import MaxPooling2D
11 from tensorflow.keras.layers import Dense
```

```

12 from tensorflow.keras.layers import Flatten
13 from tensorflow.keras.optimizers import SGD
14
15 # load train and test dataset
16 def load_dataset():
17     # load dataset
18     (trainX, trainY), (testX, testY) = mnist.load_data()
19     # reshape dataset to have a single channel
20     trainX = trainX.reshape((trainX.shape[0], 28, 28, 1))
21     testX = testX.reshape((testX.shape[0], 28, 28, 1))
22     # one hot encode target values
23     trainY = to_categorical(trainY)
24     testY = to_categorical(testY)
25     return trainX, trainY, testX, testY
26
27 # scale pixels
28 def prep_pixels(train, test):
29     # convert from integers to floats
30     train_norm = train.astype('float32')
31     test_norm = test.astype('float32')
32     # normalize to range 0-1
33     train_norm = train_norm / 255.0
34     test_norm = test_norm / 255.0
35     # return normalized images
36     return train_norm, test_norm
37
38 # define cnn model
39 def define_model():
40     model = Sequential()
41     model.add(Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_uniform', input_shape=(28, 28, 1)))
42     model.add(MaxPooling2D((2, 2)))
43     model.add(Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_uniform'))
44     model.add(Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_uniform'))
45     model.add(MaxPooling2D((2, 2)))
46     model.add(Flatten())
47     model.add(Dense(100, activation='relu', kernel_initializer='he_uniform'))
48     model.add(Dense(10, activation='softmax'))
49     # compile model
50     opt = SGD(learning_rate=0.01, momentum=0.9)
51     model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['accuracy'])
52     return model
53
54 # evaluate a model using k-fold cross-validation
55 def evaluate_model(dataX, dataY, n_folds=5):
56     scores, histories = list(), list()
57     # prepare cross validation
58     kfold = KFold(n_folds, shuffle=True, random_state=1)
59     # enumerate splits
60     for train_ix, test_ix in kfold.split(dataX):
61         # define model
62         model = define_model()
63         # select rows for train and test
64         trainX, trainY, testX, testY = dataX[train_ix], dataY[train_ix], dataX[test_ix], dataY[test_ix]
65         # fit model
66         history = model.fit(trainX, trainY, epochs=10, batch_size=32, validation_data=(testX, testY), verbose=0)
67         # evaluate model
68         _, acc = model.evaluate(testX, testY, verbose=0)
69         print('> %.3f' % (acc * 100.0))
70         # store scores
71         scores.append(acc)
72         histories.append(history)
73     return scores, histories
74
75 # plot diagnostic learning curves
76 def summarize_diagnostics(histories):
77     for i in range(len(histories)):
78         # plot loss
79         plt.subplot(2, 1, 1)
80         plt.title('Cross Entropy Loss')
81         plt.plot(histories[i].history['loss'], color='blue', label='train')
82         plt.plot(histories[i].history['val_loss'], color='orange', label='test')
83         # plot accuracy
84         plt.subplot(2, 1, 2)
85         plt.title('Classification Accuracy')
86         plt.plot(histories[i].history['accuracy'], color='blue', label='train')
87         plt.plot(histories[i].history['val_accuracy'], color='orange', label='test')
88     plt.show()
89
90 # summarize model performance
91 def summarize_performance(scores):
92     # print summary
93     print('Accuracy: mean=%.3f std=%.3f, n=%d' % (mean(scores)*100, std(scores)*100, len(scores)))
94     # box and whisker plots of results
95     plt.boxplot(scores)
96     plt.show()
97
98 # run the test harness for evaluating a model
99 def run_test_harness():
100     # load dataset
101     trainX, trainY, testX, testY = load_dataset()
102     # prepare pixel data
103     trainX, testX = prep_pixels(trainX, testX)
104     # evaluate model
105     scores, histories = evaluate_model(trainX, trainY)
106     # learning curves
107     summarize_diagnostics(histories)
108     # summarize estimated performance
109     summarize_performance(scores)
110
111 # entry point, run the test harness
112 run_test_harness()

```

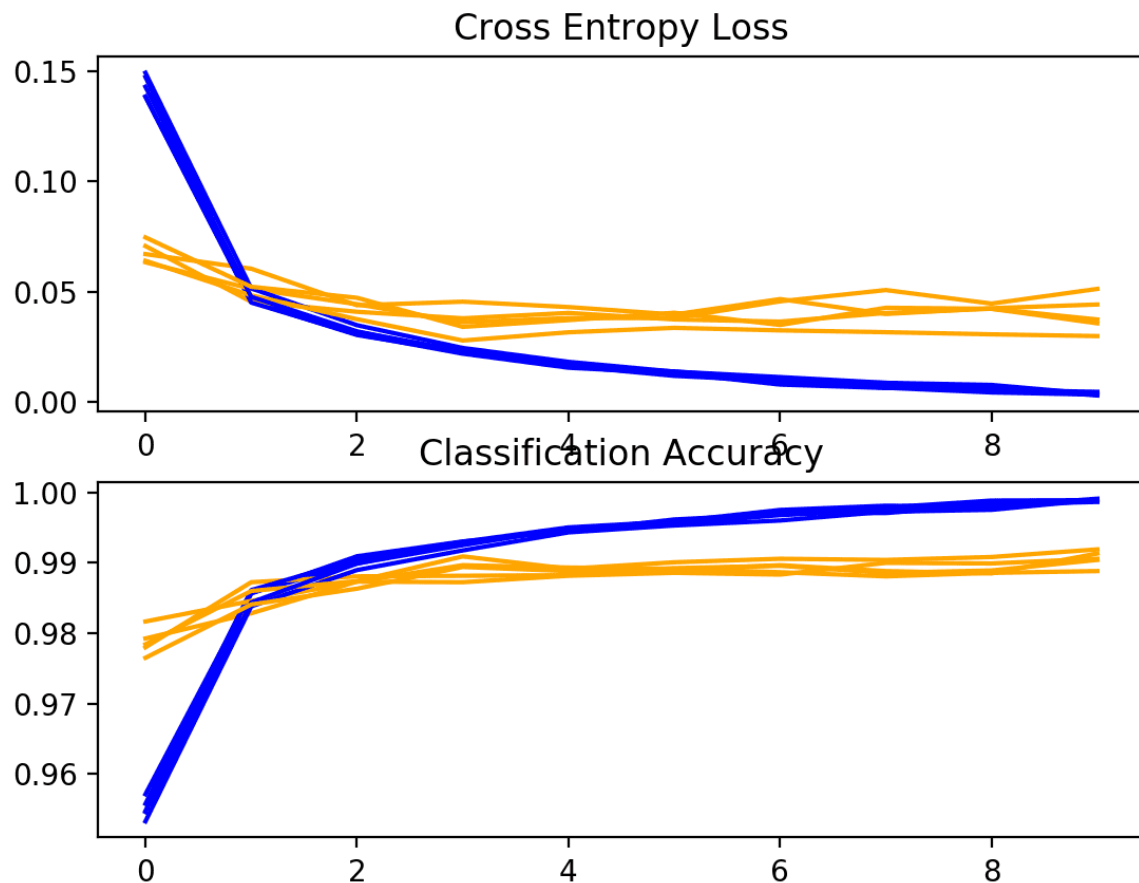
Running the example reports model performance for each fold of the cross-validation process.

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

The per-fold scores may suggest some improvement over the baseline.

```
1 > 99.058
2 > 99.042
3 > 98.883
4 > 99.192
5 > 99.133
```

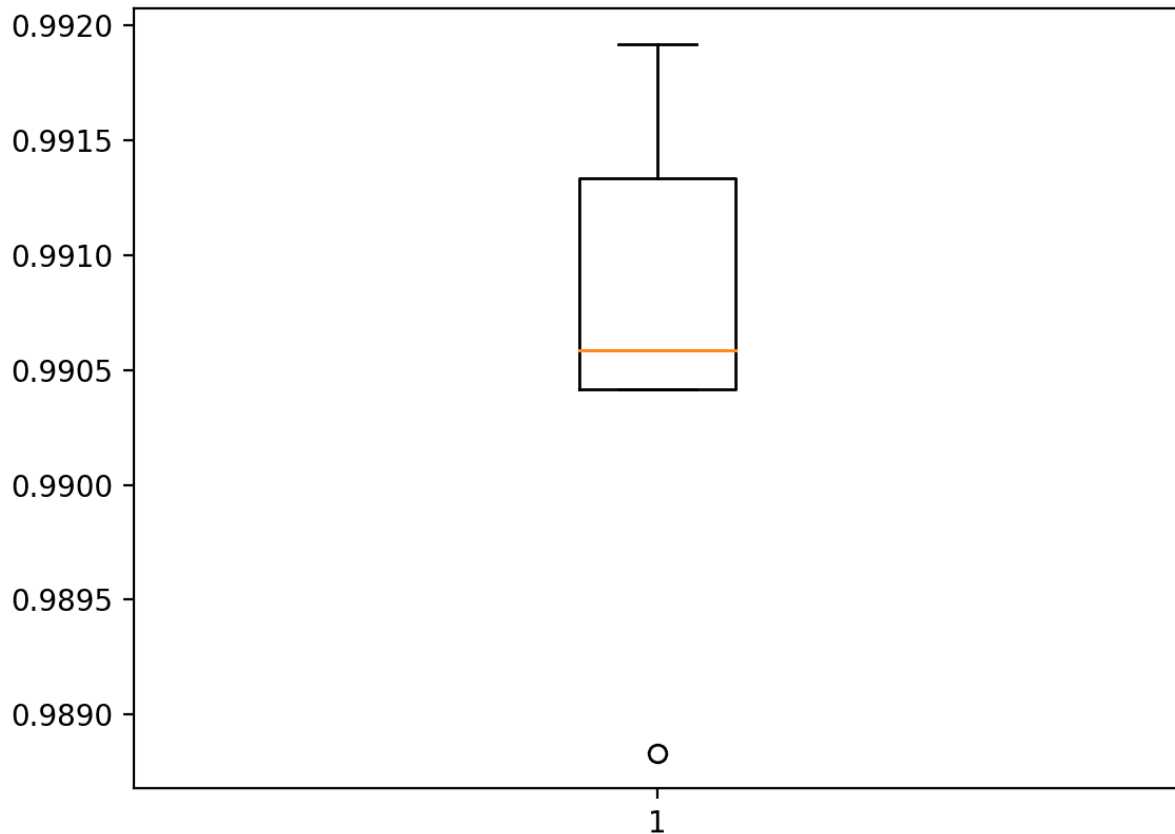
A plot of the learning curves is created, in this case showing that the models still have a good fit on the problem, with no clear signs of overfitting. The plots may even suggest that further training epochs could be helpful.



Loss and Accuracy Learning Curves for the Deeper Model During k-Fold Cross-Validation

Next, the estimated performance of the model is presented, showing a small improvement in performance as compared to the baseline from 98.677 to 99.062, with a small drop in the standard deviation as well.

```
1 Accuracy: mean=99.062 std=0.104, n=5
```



Box and Whisker Plot of Accuracy Scores for the Deeper Model Evaluated Using k-Fold Cross-Validation

How to Finalize the Model and Make Predictions

The process of model improvement may continue for as long as we have ideas and the time and resources to test them out.

At some point, a final model configuration must be chosen and adopted. In this case, we will choose the deeper model as our final model.

First, we will finalize our model, but fitting a model on the entire training dataset and saving the model to file for later use. We will then load the model and evaluate its performance on the hold out test dataset to get an idea of how well the chosen model actually performs in practice. Finally, we will use the saved model to make a prediction on a single image.

Save Final Model

A final model is typically fit on all available data, such as the combination of all train and test dataset.

In this tutorial, we are intentionally holding back a test dataset so that we can estimate the performance of the final model, which can be a good idea in practice. As such, we will fit our model on the training dataset only.

```
1 # fit model
2 model.fit(trainX, trainY, epochs=10, batch_size=32, verbose=0)
```

Once fit, we can save the final model to an H5 file by calling the `save()` function on the model and pass in the chosen filename.

```
1 # save model
2 model.save('final_model.h5')
```

Note, saving and loading a Keras model requires that the [h5py library](#) is installed on your workstation.

The complete example of fitting the final deep model on the training dataset and saving it to file is listed below.

```
1 # save the final model to file
2 from tensorflow.keras.datasets import mnist
3 from tensorflow.keras.utils import to_categorical
4 from tensorflow.keras.models import Sequential
5 from tensorflow.keras.layers import Conv2D
6 from tensorflow.keras.layers import MaxPooling2D
7 from tensorflow.keras.layers import Dense
8 from tensorflow.keras.layers import Flatten
```



```

9  from tensorflow.keras.optimizers import SGD
10
11 # load train and test dataset
12 def load_dataset():
13     # load dataset
14     (trainX, trainY), (testX, testY) = mnist.load_data()
15     # reshape dataset to have a single channel
16     trainX = trainX.reshape((trainX.shape[0], 28, 28, 1))
17     testX = testX.reshape((testX.shape[0], 28, 28, 1))
18     # one hot encode target values
19     trainY = to_categorical(trainY)
20     testY = to_categorical(testY)
21     return trainX, trainY, testX, testY
22
23 # scale pixels
24 def prep_pixels(train, test):
25     # convert from integers to floats
26     train_norm = train.astype('float32')
27     test_norm = test.astype('float32')
28     # normalize to range 0-1
29     train_norm = train_norm / 255.0
30     test_norm = test_norm / 255.0
31     # return normalized images
32     return train_norm, test_norm
33
34 # define cnn model
35 def define_model():
36     model = Sequential()
37     model.add(Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_uniform', input_shape=(28, 28, 1)))
38     model.add(MaxPooling2D((2, 2)))
39     model.add(Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_uniform'))
40     model.add(Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_uniform'))
41     model.add(MaxPooling2D((2, 2)))
42     model.add(Flatten())
43     model.add(Dense(100, activation='relu', kernel_initializer='he_uniform'))
44     model.add(Dense(10, activation='softmax'))
45     # compile model
46     opt = SGD(learning_rate=0.01, momentum=0.9)
47     model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['accuracy'])
48     return model
49
50 # run the test harness for evaluating a model
51 def run_test_harness():
52     # load dataset
53     trainX, trainY, testX, testY = load_dataset()
54     # prepare pixel data
55     trainX, testX = prep_pixels(trainX, testX)
56     # define model
57     model = define_model()
58     # fit model
59     model.fit(trainX, trainY, epochs=10, batch_size=32, verbose=0)
60     # save model
61     model.save('final_model.h5')
62
63 # entry point, run the test harness
64 run_test_harness()

```

After running this example, you will now have a 1.2-megabyte file with the name `'final_model.h5'` in your current working directory.

Evaluate Final Model

We can now load the final model and evaluate it on the hold out test dataset.

This is something we might do if we were interested in presenting the performance of the chosen model to project stakeholders.

The model can be loaded via the `load_model()` function.

The complete example of loading the saved model and evaluating it on the test dataset is listed below.

```

1  # evaluate the deep model on the test dataset
2  from tensorflow.keras.datasets import mnist
3  from tensorflow.keras.models import load_model
4  from tensorflow.keras.utils import to_categorical
5
6  # load train and test dataset
7  def load_dataset():
8      # load dataset
9      (trainX, trainY), (testX, testY) = mnist.load_data()
10     # reshape dataset to have a single channel
11     trainX = trainX.reshape((trainX.shape[0], 28, 28, 1))
12     testX = testX.reshape((testX.shape[0], 28, 28, 1))
13     # one hot encode target values
14     trainY = to_categorical(trainY)
15     testY = to_categorical(testY)
16     return trainX, trainY, testX, testY
17
18 # scale pixels
19 def prep_pixels(train, test):
20     # convert from integers to floats
21     train_norm = train.astype('float32')
22     test_norm = test.astype('float32')
23     # normalize to range 0-1
24     train_norm = train_norm / 255.0
25     test_norm = test_norm / 255.0
26     # return normalized images
27     return train_norm, test_norm
28
29 # run the test harness for evaluating a model
30 def run_test_harness():

```

```

31 # load dataset
32 trainX, trainY, testX, testY = load_dataset()
33 # prepare pixel data
34 trainX, testX = prep_pixels(trainX, testX)
35 # load model
36 model = load_model('final_model.h5')
37 # evaluate model on test dataset
38 _, acc = model.evaluate(testX, testY, verbose=0)
39 print('> %.3f' % (acc * 100.0))
40
41 # entry point, run the test harness
42 run_test_harness()

```

Running the example loads the saved model and evaluates the model on the hold out test dataset.

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

The classification accuracy for the model on the test dataset is calculated and printed. In this case, we can see that the model achieved an accuracy of 99.090%, or just less than 1%, which is not bad at all and reasonably close to the estimated 99.753% with a standard deviation of about half a percent (e.g. 99% of scores).

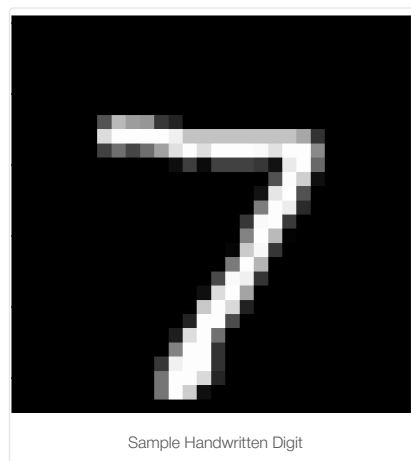
```
1 > 99.090
```

Make Prediction

We can use our saved model to make a prediction on new images.

The model assumes that new images are grayscale, that they have been aligned so that one image contains one centered handwritten digit, and that the size of the image is square with the size 28×28 pixels.

Below is an image extracted from the MNIST test dataset. You can save it in your current working directory with the filename `'sample_image.png'`.



- Download the sample image (sample_image.png)

We will pretend this is an entirely new and unseen image, prepared in the required way, and see how we might use our saved model to predict the integer that the image represents (e.g. we expect “7”).

First, we can load the image, force it to be in grayscale format, and force the size to be 28×28 pixels. The loaded image can then be resized to have a single channel and represent a single sample in a dataset. The `load_image()` function implements this and will return the loaded image ready for classification.

Importantly, the pixel values are prepared in the same way as the pixel values were prepared for the training dataset when fitting the final model, in this case, normalized.

```

1 # load and prepare the image
2 def load_image(filename):
3     # load the image
4     img = load_img(filename, grayscale=True, target_size=(28, 28))
5     # convert to array
6     img = img_to_array(img)
7     # reshape into a single sample with 1 channel
8     img = img.reshape(1, 28, 28, 1)
9     # prepare pixel data
10    img = img.astype('float32')
11    img = img / 255.0
12    return img

```

Next, we can load the model as in the previous section and call the `predict()` function to get the predicted score, and then use `argmax()` to obtain the digit that the image represents.

```

1 # predict the class
2 predict_value = model.predict(img)
3 digit = argmax(predict_value)

```

The complete example is listed below.

```
1 # make a prediction for a new image.
2 from numpy import argmax
3 from keras.preprocessing.image import load_img
4 from keras.preprocessing.image import img_to_array
5 from keras.models import load_model
6
7 # load and prepare the image
8 def load_image(filename):
9     # load the image
10    img = load_img(filename, grayscale=True, target_size=(28, 28))
11    # convert to array
12    img = img_to_array(img)
13    # reshape into a single sample with 1 channel
14    img = img.reshape(1, 28, 28, 1)
15    # prepare pixel data
16    img = img.astype('float32')
17    img = img / 255.0
18    return img
19
20 # load an image and predict the class
21 def run_example():
22     # load the image
23     img = load_image('sample_image.png')
24     # load model
25     model = load_model('final_model.h5')
26     # predict the class
27     predict_value = model.predict(img)
28     digit = argmax(predict_value)
29     print(digit)
30
31 # entry point, run the example
32 run_example()
```

Running the example first loads and prepares the image, loads the model, and then correctly predicts that the loaded image represents the digit '7'.

1 7

Extensions

This section lists some ideas for extending the tutorial that you may wish to explore.

- **Tune Pixel Scaling.** Explore how alternate pixel scaling methods impact model performance as compared to the baseline model, including centering and standardization.
- **Tune the Learning Rate.** Explore how different learning rates impact the model performance as compared to the baseline model, such as 0.001 and 0.0001.
- **Tune Model Depth.** Explore how adding more layers to the model impact the model performance as compared to the baseline model, such as another block of convolutional and pooling layers or another dense layer in the classifier part of the model.

If you explore any of these extensions, I'd love to know.

Post your findings in the comments below.

Further Reading

This section provides more resources on the topic if you are looking to go deeper.

APIs

- [Keras Datasets API](#)
- [Keras Datasets Code](#)
- [sklearn.model_selection.KFold API](#)

Articles

- [MNIST database, Wikipedia.](#)
- [Classification datasets results, What is the class of this image?](#)

Summary

In this tutorial, you discovered how to develop a convolutional neural network for handwritten digit classification from scratch.

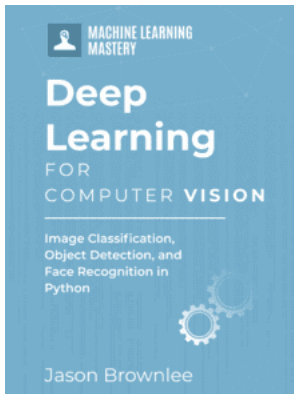
Specifically, you learned:

- How to develop a test harness to develop a robust evaluation of a model and establish a baseline of performance for a classification task.
- How to explore extensions to a baseline model to improve learning and model capacity.
- How to develop a finalized model, evaluate the performance of the final model, and use it to make predictions on new images.

Do you have any questions?

Ask your questions in the comments below and I will do my best to answer.

Develop Deep Learning Models for Vision Today!



Develop Your Own Vision Models in Minutes

...with just a few lines of python code

Discover how in my new Ebook:
Deep Learning for Computer Vision

It provides **self-study tutorials** on topics like:
classification, object detection (yolo and rcnn), face recognition (vggface and facenet), data preparation and much more...

Finally Bring Deep Learning to your Vision Projects

Skip the Academics. Just Results.

SEE WHAT'S INSIDE

Share Share Post

More On This Topic



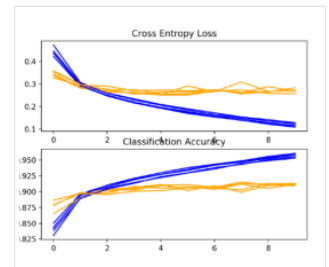
How to Develop a GAN for Generating MNIST Handwritten Digits



Handwritten Digit Recognition Using Convolutional...



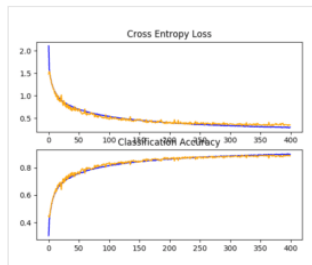
Handwritten Digit Recognition with LeNet5 Model in PyTorch



Deep Learning CNN for Fashion-MNIST Clothing Classification



How to Develop a Multichannel CNN Model for Text...



How to Develop a CNN From Scratch for CIFAR-10 Photo...



About Jason Brownlee

Jason Brownlee, PhD is a machine learning specialist who teaches developers how to get results with modern machine learning methods via hands-on tutorials.
[View all posts by Jason Brownlee →](#)

< [How to Visualize Filters and Feature Maps in Convolutional Neural Networks](#)

[Deep Learning CNN for Fashion-MNIST Clothing Classification](#) >

193 Responses to *How to Develop a CNN for MNIST Handwritten Digit Classification*



SHAHEEN ALHIRMIZY May 10, 2019 at 4:32 pm #

REPLY ↩